راهنمای مصاحبه‌گر

جلسه‌ی همراه با هوش مصنوعی: MiA Cold Start

این جلسه چه چیزی را می‌سنجد

یک جلسه‌ی دوساعته روی کدی که داوطلب هرگز ندیده، با این تفاوت که از او انتظار می‌رود از یک دستیار هوش مصنوعی استفاده کند. سؤال این نیست که تمام می‌کند یا نه. سؤال این است که با دستیار، داخل کدی ناآشنا، چطور کار می‌کند و آیا متوجه می‌شود دستیار کجا اشتباه می‌گوید.

این ابزار با «جلسه‌ی کد» فرق دارد. جلسه‌ی کد نشان می‌دهد کسی از صفحه‌ی خالی چطور می‌سازد. این جلسه نشان می‌دهد وقتی بیشترِ تایپ‌کردن را یک ابزار انجام می‌دهد، چطور می‌خواند، راستی‌آزمایی می‌کند و جهت می‌دهد.

چرا این کد

MiA یک میان‌افزار کوچک اینترنت اشیا است: عامل‌ها (agent) داده‌ی دستگاه‌ها را روی MQTT منتشر می‌کنند و کاربر از طریق HTTP آن را می‌خواند و دستگاه را کنترل می‌کند. حدود ۱۹۰۰ خط پایتون در ۷۱ فایل، شش تست که همه سبزند.

آن‌قدر کوچک است که در یک ساعت خوانده شود، اما روی ویژگی‌هایی از پایتون بنا شده که اکثر افراد نصفه‌نیمه به یاد دارند: یک metaclass که هر زیرکلاس `Thing` را با نام ثبت می‌کند، descriptorهایی با اثر جانبی (انتساب به `Setting` یک پیام MQTT منتشر می‌کند و انتساب به `State` در Mongo می‌نویسد)، کنترلرهای singleton که با class attribute از `main.py` سیم‌کشی شده‌اند، و بارگذاری تنبل نوعِ دستگاه‌ها با `importlib` بر اساس نام ماژول. دستیار هوش مصنوعی همه‌ی این‌ها را روان توضیح می‌دهد و در جزئیات اشتباه می‌کند. همین ویژگی را می‌خواهیم.

کد یک runtime واقعی هم دارد: بروکر MQTT، پایگاه داده‌ی Mongo، یک فرایند عامل و یک API. داوطلب می‌تواند ادعاها را سرتاسری آزمایش کند، نه فقط بخواند. این‌که انتخاب می‌کند آزمایش کند یا نه، اصلی‌ترین چیزی است که مشاهده می‌کنیم.

شاخه‌ی `main` در کامیت `f0bd540` پنج باگ پنهان دارد که CI آن‌ها را نمی‌گیرد. همه پیش از نوشتن این جلسه با نسخه‌های قفل‌شده‌ی کتابخانه‌ها بازتولید شده‌اند؛ هیچ‌کدام ساختگی نیست.

باگ	کجا	اثر	تسک
تعداد آرگومان callback در paho-mqtt 2.x	`I1820/mqtt/route.py`	نخ شبکه روی CONNACK می‌میرد؛ هیچ topicی subscribe نمی‌شود	۰
timestamp پیش‌فرضِ منجمد	`I1820/domain/log.py`	همه‌ی لاگ‌های عامل آزمایشی یک timestamp دارند: لحظه‌ی import	۰
فهرست فیلدهای مشترک بین همه‌ی نوع‌ها	`I1820/things/base.py`	هر نوع، فیلدهای همه‌ی نوع‌های بارشده را گزارش می‌کند	۲
`last_seen` هیچ‌وقت تازه نمی‌شود	`I1820/discovery/discovery.py`	زمان عامل روی اولین ping می‌ماند	۳
خواندن settingی که مقدار نگرفته	`I1820/things/fields/setting.py`	`KeyError` به HTTP 500 تبدیل می‌شود	۳

قالب جلسه

فردی، ۱۲۰ دقیقه چهار تسک زمان‌بندی‌شده و یک تسک اضافه، بعد ۱۵ دقیقه جمع‌بندی.

هوش مصنوعیِ خودش هر ابزاری آزاد است؛ Copilot، Cursor، Claude Code یا یک پنجره‌ی چت. ثبت کنید کدام ابزار بود و آیا خودش می‌توانست دستور اجرا کند. پنجره‌ی چت داوطلب را مجبور می‌کند خودش دست‌به‌کار شود؛ ابزار agentic سؤال را به این‌جا می‌برد که آیا داوطلب می‌خواند ابزار چه اجرا کرد.

با ناظر یک مصاحبه‌گر با برگه‌ی یادداشت و ضبط صفحه. اگر ناظر نیست، متن گفت‌وگو با هوش مصنوعی و تاریخچه‌ی git بیشترِ سیگنال را نگه می‌دارد.

کامیت بعد از هر تسک داوطلب انتهای هر تسک را تگ می‌کند (`task-0`، `task-1`، …). کار ناتمام قابل قبول است و خودش شاهد است.

تسک‌ها یکی‌یکی داده می‌شوند تسک‌های بعدی به باگ‌هایی اشاره می‌کنند که تسک‌های قبلی ممکن است رو کرده باشند؛ زود دادنشان سیگنال را خراب می‌کند.

محیط از قبل آماده است ایمیج‌ها pull شده، وابستگی‌ها نصب شده. اصطکاک راه‌اندازی نویز است، نه سیگنال.

آماده‌سازی

یک fork با نامی خنثی (مثلاً `mia-challenge`) بسازید. کل کار کمتر از یک ساعت است.

- [ ] fork در کامیت `f0bd540` و حذف پوشه‌ی `.github/` تا کسی به badge سبزِ CI اعتماد نکند. README را نگه دارید؛ پروتکل MQTT را توضیح می‌دهد و بخشی از آزمون است.
- [ ] باگ paho را درست **نکنید**. تسک ۰ به آن وابسته است.
- [ ] شمارنده‌ی عامل آزمایشی را افزایشی کنید تا «آخرین مقدار» قابل مشاهده باشد. فراخوانی `I1820Log(...)` را بدون timestamp صریح نگه دارید؛ تله‌ی دوم همین است.
- [ ] ایمیج‌های `mongo:5.0` و `emqx/emqx:5.0.0` را pull کنید و یک محیط پایتون ۳.۱۲ با وابستگی‌های قفل‌شده آماده کنید.
- [ ] روی ماشین آماده‌سازی مطمئن شوید تله کار می‌کند: `main.py` و `dummy.py` را اجرا کنید، `GET /agents` باید `{}` برگرداند و traceback خطای `TypeError` مربوط به paho در ترمینال سرور دیده شود.
- [ ] این راهنما را چاپ کنید و صورت هر تسک را روی کارت جدا داشته باشید.

تغییر `dummy.py`:

```
-    i = 2
+    i = 0
     while True:
-        i = (i**2) % 1024
+        i += 1
         ping()
```

متنی که به داوطلب می‌دهید

بدون تغییر بخوانید: شما به تیمی می‌پیوندید که MiA، یک میان‌افزار کوچک اینترنت اشیا، را نگه‌داری می‌کند. عامل‌ها داده‌ی دستگاه‌ها را روی MQTT منتشر می‌کنند و کاربران از طریق HTTP آن‌ها را می‌خوانند و کنترل می‌کنند. شما این کد را هرگز ندیده‌اید. دو ساعت وقت و یک دستیار هوش مصنوعی به انتخاب خودتان دارید.

۱. هر قدر که می‌خواهید از هوش مصنوعی استفاده کنید. برای ما مهم است چطور کار می‌کنید، نه این‌که از آن استفاده می‌کنید یا نه. ۲. بلند فکر کنید. بگویید از هوش مصنوعی چه می‌پرسید و چرا، و بعد از جوابش چه باوری دارید. ۳. بعد از هر تسک کامیت و تگ کنید (`git tag task-N`). ناتمام هم قابل قبول است. ۴. اگر چیزی در تسک مبهم است می‌توانید از ما بپرسید. ممکن است جواب بدهیم، ممکن است ندهیم. ۵. متن گفت‌وگو با هوش مصنوعی را نگه دارید. در انتها جمع می‌کنیم.

TASK 0 روشن‌شدن اولین چراغ

۱۵ دقیقه

صورت مسئله برای داوطلب

سیستم را بالا بیاورید (`docker compose up -d`، بعد `main.py` و `dummy.py`). به ما نشان دهید:

- `GET /agents` عاملی به نام `dummy` با یک دستگاه از نوع `dummy` را فهرست می‌کند.
- `POST /things` برای آن دستگاه مقدار `state` را برمی‌گرداند، و یک فراخوانی دوم چند ثانیه بعد مقدارِ **بزرگ‌تری** برمی‌گرداند.

تله‌ی ۱ · سرور می‌گوید بالاست، ولی کر است. handler برای امضای callback نسخه‌ی x.1 کتابخانه‌ی paho نوشته شده، اما client با `CallbackAPIVersion.VERSION2` ساخته می‌شود. روی CONNACK، paho تابع `on_connect` را با پنج آرگومان صدا می‌زند، متد چهار تا می‌پذیرد و نخ شبکه با این خطا می‌میرد:

```
TypeError: Handler.on_connect() takes 5 positional arguments but 6 were given
```

traceback بین خط‌های لاگ rich چاپ می‌شود، بعد «Mia is up and running» می‌آید و سرور HTTP عادی کار می‌کند. `GET /agents` جواب `{}` می‌دهد. هیچ subscribeای هرگز انجام نمی‌شود.

تله‌ی ۲ · مقدار هیچ‌وقت تازه‌تر نمی‌شود. سازنده‌ی `I1820Log` آرگومان پیش‌فرض `timestamp=datetime.utcnow()` دارد که یک بار در لحظه‌ی import ارزیابی می‌شود. عامل آزمایشی هیچ‌وقت timestamp نمی‌دهد، پس همه‌ی لاگ‌هایش یک زمان دارند. `find_one(sort=time desc)` در Mongo بین مقادیر مساوی یکی را برمی‌گرداند و «آخرین» بی‌معنا می‌شود. معیار دوم تا فهمیدن این موضوع برآورده نمی‌شود.

به چه چیزی نگاه کنید:

- ترمینال سرور را می‌خواند، یا «agents خالی است» را بدون traceback به هوش مصنوعی می‌دهد؟
- وقتی هوش مصنوعی شبکه‌ی Docker، tenant یا بروکر را مقصر می‌داند، فرضیه را آزمایش می‌کند یا تغییر پیشنهادی را کور اعمال می‌کند؟

- برای تله‌ی ۲، هوش مصنوعی معمولاً پیشنهاد می‌دهد به‌جای `time` روی `_id` مرتب شود. این علامت را می‌پوشاند و کار هم می‌کند. داوطلب می‌پرسد **چرا** timestampها مساوی‌اند، یا وصله‌ی کارکننده را می‌پذیرد؟
- بعد از هر اصلاح، کل حلقه را دوباره اجرا می‌کند یا با اولین جواب غیرخالی اعلام موفقیت می‌کند؟

TASK 1 ردگیری یک لاگ

۱۵ دقیقه

صورت مسئله برای داوطلب

در حداکثر ده خط بنویسید از لحظه‌ای که یک عامل روی `I1820/main/log/send` منتشر می‌کند تا لحظه‌ای که کاربر آن مقدار را با `POST /things` می‌خواند، چه اتفاقی می‌افتد. در هر مرحله نام فایل و تابع را بنویسید. بعد جواب دهید: مقدار بین این دو لحظه کجا زندگی می‌کند، و اگر عامل هیچ‌وقت ping نفرستاده بود چه می‌شد؟

این تسک تله ندارد. خط پایه‌ی این است که داوطلب برای فهمیدن چطور از هوش مصنوعی استفاده می‌کند، و او را برای تسک ۲ که به فهم metaclass وابسته است آماده می‌کند. زنجیره‌ی درست:

۱. `Handler.on_log` در `mqtt/route.py` با `I1820Log.from_json` ورودی را با `log.json` اعتبارسنجی می‌کند.
۲. `Things.get(type)` در `things/base.py` ماژول `I1820.things.models.<type>` را در لحظه import می‌کند؛ metaclass همان‌جا کلاس را ثبت کرده است.
۳. `get_thing(agent, device)` نمونه را در `registered_things` پیدا می‌کند که `DiscoveryService.ping` پر کرده. بدون ping، `ThingNotFoundException` و لاگ با یک warning دور انداخته می‌شود.
۴. `setattr(thing, name, {...})` به `State.__set__` می‌رسد که singleton `LogController` را صدا می‌زند؛ `log_service` آن در `main.py` به‌عنوان class attribute تزریق شده و در نهایت به `MongodbLogAppender.create` می‌رسد.
۵. در خواندن، `getattr` به `State.__get__` می‌رسد که `retrieve_last` را از Mongo می‌گیرد. مقدار فقط در Mongo زندگی می‌کند؛ نمونه‌ی thing هیچ چیزی نگه نمی‌دارد.

به چه چیزی نگاه کنید:

- توضیح هوش مصنوعی را همان‌طور می‌پذیرد، یا از آن ارجاع به خط می‌خواهد و فایل‌ها را خودش باز می‌کند؟
- اشتباه‌های رایج هوش مصنوعی این‌جا: ادعای این‌که state روی نمونه cache می‌شود؛ توصیف `Things.get` به‌عنوان خواندن یک فایل registry؛ ندیدن این‌که `LogController` یک singleton است که از `main.py` سیم‌کشی شده. بشمارید چند تا را می‌گیرد.
- از runtime برای تأیید استفاده می‌کند؟ مثلاً `dummy.py` را متوقف کند و سرور را دوباره بالا بیاورد تا warning دورانداختن لاگ را ببیند.

TASK 2 لامپ قابل تنظیم

۳۰ دقیقه

صورت مسئله برای داوطلب

نوع دستگاه `dimmable_lamp` را اضافه کنید. هر چه `lamp` دارد را دارد، به‌اضافه‌ی setting `brightness` (عدد صحیح، ۰ تا ۱۰۰) و statistic `power`. معیار پذیرش:

- `GET /models/dimmable_lamp` دقیقاً فیلدهای خودش را توصیف می‌کند: settingهای `on` و `brightness`، statistic `power`، و هیچ چیز دیگر.
- `PUT /things` با `brightness: 60` یک notification روی `I1820/main/configuration/request` منتشر می‌کند؛ با هر subscriber MQTT نشانش دهید.
- تست هر دو را پوشش می‌دهد.

تله · زیرکلاسِ بدیهی درست است، و endpoint هنوز غلط جواب می‌دهد. `class DimmableLamp(Lamp)` با دو فیلد جدید شکل درستی است. اما `AbstractThing.__new__` فیلدها را با `getattr(instance, field_name).append(value)` جمع می‌کند و فهرست‌هایی که به آن‌ها اضافه می‌کند همان‌هایی‌اند که روی خودِ `Thing` تعریف شده‌اند. همه‌ی نوع‌ها در آن‌ها مشترک‌اند. وقتی `curtain`، `lamp` و `temperature` import شده باشند، endpoint این را گزارش می‌کند:

```
{"type": "dimmable_lamp", "settings": ["on", "height", "brightness"],
 "states": ["temperature"], "statistics": ["power"], "events": []}
```

همین باگ `POST /things` با فهرست `states` خالی را روی هر actuator خراب می‌کند، چون handler بعدش `getattr(lamp, "temperature")` را صدا می‌زند و ۵۰۰ می‌دهد. اصلاح درست در metaclass است: به هر کلاس فهرست‌های خودش را بدهد که از کلاس‌های پایه‌اش شروع می‌شوند. فیلترکردن در `to_dict` علامت را فقط برای یک endpoint می‌پوشاند.

تله‌ی کوچک‌تر: فایل باید `models/dimmable_lamp.py` باشد، چون `Things.get` با نام ماژول import می‌کند و بعد نام کلاس را به‌شکل snake_case جست‌وجو می‌کند. `dimmablelamp.py` بدون خطا import می‌شود و بعد `ThingTypeNotImplementedException` گمراه‌کننده‌ای می‌دهد.

به چه چیزی نگاه کنید:

- بعد از این‌که هوش مصنوعی گفت تمام شد، واقعاً `GET /models/dimmable_lamp` را صدا می‌زند؟ بیشترِ اجراها کلاس را می‌نویسند، شاید تستی که `issubclass` را چک کند، و متوقف می‌شوند.
- وقتی فیلدهای اضافه را دید، بررسی را به نوع‌های موجود گسترش می‌دهد یا فقط نوع جدید را درست می‌کند؟
- ریشه یا علامت: اصلاح metaclass، فیلتر در `to_dict`، یا فهرست hardcode؟
- اعتبارسنجی بازه‌ی brightness کجا می‌رود؟ هوش مصنوعی معمولاً آن را در handler HTTP می‌گذارد. داوطلب می‌پرسد descriptor جای درست‌تری نیست، وقتی `Setting.__set__` پیش از هر اعتبارسنجی منتشر می‌کند؟
- برای تستِ notification، فایل `test_thing.py` الگوی `DummyMQTTService` را برای singleton نشان می‌دهد. هوش مصنوعی آن را پیدا می‌کند یا mock تازه‌ای می‌سازد، و داوطلب تکرار را می‌بیند؟

TASK 3 دو گزارش باگ

۲۵ دقیقه

صورت مسئله برای داوطلب

دو تیکت از تیم عملیات. هر دو را با تست رگرسیونی درست کنید که پیش از اصلاح شکست بخورد و بعد از آن بگذرد.

- **OPS-118**: «فیلد `time` در `GET /agents` برای عاملی که ساعت‌ها ping می‌فرستد هیچ‌وقت تغییر نمی‌کند. alert عامل‌های stale ما هرگز فعال نمی‌شود.»
- **OPS-121**: «خواندن یک لامپ با `POST /things` و `settings: ["on"]` اگر کسی هنوز آن را روشن یا خاموش نکرده باشد HTTP 500 می‌دهد. جواب تمیزی می‌خواهیم.»

تله · دو خطِ غلط کنارِ خط درست. در `DiscoveryService.ping` شاخه‌ی ping دوباره `agent.last_seen = now` را روی شیء ورودی انجام می‌دهد که بعدش دور انداخته می‌شود. عامل ذخیره‌شده اولین timestamp خودش را نگه می‌دارد. دو خط بالاتر `if self._agents[agent.ident] == agent.things` نشسته که یک Agent را با یک set مقایسه می‌کند و همیشه نادرست است. هوش مصنوعی اغلب همان مقایسه را «درست» می‌کند و اعلام موفقیت می‌کند، در حالی که `last_seen` هنوز کهنه است. تست موجود `test_discovery.py` در تمام این مدت پاس می‌شود.

برای `Setting.__get__`، OPS-121 یک lookup بدون محافظ روی dict دارد. README می‌گوید settingها پیش از خواندن باید مقدار گرفته باشند، پس قرارداد یک تصمیم واقعی است: برگرداندن `null` مثل `Statistic`، یا ۴۰۴، یا ۴۰۰. داوطلب باید متوجه شود این تصمیم فقط مال او نیست. برگرداندن None نیاز دارد نوع بازگشتی به `T | None` تغییر کند، وگرنه mypy در CI شکست می‌خورد.

به چه چیزی نگاه کنید:

- تست جدید را **پیش از** اصلاح اجرا می‌کند تا شکستش را ببیند؟ تستی که هوش مصنوعی می‌نویسد و دو ping بدون جلو بردن زمان می‌سازد، در هر دو حالت پاس می‌شود.
- اصلاحِ خطِ غلط را می‌گیرد، و چطور: با خواندن diff، با اجرای سرور، یا اصلاً نه؟
- سؤال قرارداد API را با مصاحبه‌گر مطرح می‌کند، یا می‌گذارد هوش مصنوعی بی‌سروصدا انتخاب کند؟
- اصلاح OPS-121 در descriptor می‌ماند، یا هوش مصنوعی handler را در try/except می‌پیچد که `KeyError` بعدی را هم پنهان می‌کند؟

TASK 4 چند دستگاه (اضافه)

اگر وقت ماند

صورت مسئله برای داوطلب

`POST /things` فهرستی از `device_id` می‌پذیرد اما یک شیء تخت برمی‌گرداند، پس فقط مقادیر آخرین دستگاه زنده می‌ماند. کاری کنید نتیجه به‌تفکیک دستگاه برگردد بدون این‌که فراخوانی تک‌دستگاهی بشکند.

این تسک بیشتر طراحی است تا باگ‌یابی و تله ندارد. نکته این است که داوطلب سازگاری با گذشته را در نظر می‌گیرد و درباره‌ی شکل پاسخ می‌پرسد، یا می‌گذارد هوش مصنوعی یکی را انتخاب کند. برای پر کردن وقت باقی‌مانده و دیدن رفتارش وقتی جواب «درستی» برای راستی‌آزمایی وجود ندارد.

سنجش: با همان چهار شایستگی

هر تسک با **همان چهار شایستگی** و **همان مقیاس** سند ارزیابی سنجیده می‌شود: **ضعیف · مرزی · قوی · سنجیده نشد**. هوش مصنوعی شایستگی‌ها را تغییر نمی‌دهد؛ چیزی را تغییر می‌دهد که به‌عنوان شاهد هر شایستگی حساب می‌شود.

شایستگی	ضعیف	قوی
داده‌ساختار، الگوریتم و حل مسئله	جایی وصله می‌زند که علامت ظاهر شده (_id، فیلتر در to_dict)؛ در جمع‌بندی نمی‌تواند بگوید state کجا زندگی می‌کند یا metaclass چه می‌کند	ریشه‌ی هر باگ را پیدا می‌کند و اثرش بر جاهای دیگر را چک می‌کند؛ جریان داده را بدون کمک می‌کشد
کدنویسی	اجازه می‌دهد هوش مصنوعی metaclass را بازنویسی کند، pydantic اضافه کند یا به FastAPI مهاجرت کند؛ تست پیش از اصلاح هم پاس می‌شود	diff را محدود به تسک نگه می‌دارد؛ اول تست شکست‌خورده می‌نویسد؛ الگوهای موجود مخزن را دنبال می‌کند
ارتباط مؤثر	پرامپت‌هایی مثل «agents خالیه، چرا»؛ توضیح هوش مصنوعی را همان‌طور می‌پذیرد؛ قرارداد API را هوش مصنوعی بی‌صدا تصمیم می‌گیرد	پرامپت traceback، فایل و انتظار را دارد؛ توضیح غلط را با شاهد به چالش می‌کشد؛ تصمیم طراحی را نام می‌برد و می‌پرسد یا فرضش را می‌گوید
سرعت و به‌نتیجه‌رسانی	پیشنهادهای هوش مصنوعی را یکی‌یکی اعمال می‌کند تا چیزی کار کند؛ هیچ‌وقت endpoint را صدا نمی‌زند یا تست نمی‌گیرد	پیش از اصلاح بازتولید و بعد از آن راستی‌آزمایی می‌کند، diff را می‌خواند، و همچنان در بودجه‌ی زمانی به کد کارکننده می‌رسد

سه شمارش ساده هم برای هر تسک ثبت کنید: **فایل‌هایی که داوطلب باز کرد**، **فایل‌هایی که هوش مصنوعی باز کرد**، و **ادعاهای هوش مصنوعی که بدون بررسی پذیرفته شد**. این سه، «کد می‌خواند» را از «درباره‌ی کد می‌خواند» بهتر از هر نمره‌ای جدا می‌کنند.

جمع‌آوری داده و جمع‌بندی

جمع کنید تاریخچه‌ی git با تگ‌های task-N و working tree پایانی؛ متن گفت‌وگو با هوش مصنوعی (خروجی جلسه‌ی Claude Code، چت Cursor، یا متن پنجره‌ی چت)؛ ضبط صفحه با صدای بلندفکرکردن؛ برگه‌ی مصاحبه‌گر با یادداشت‌های زمان‌دار.

جمع‌بندی، ۱۵ دقیقه

۱. مسیر یک پیام لاگ را از عامل تا Mongo بکش. کدام فایل نوع دستگاه را ثبت می‌کند، و کِی؟
۲. کدام ادعاهای هوش مصنوعی امروز غلط از آب درآمد؟ چطور فهمیدی؟
۳. لحظه‌ای بود که اعتمادت به هوش مصنوعی را از دست بدهی؟ چه چیزی باعثش شد؟
۴. کدام تغییر را کمتر از همه راحت merge می‌کنی، و چرا؟
۵. دو سرور MiA با tenant متفاوت روی یک بروکر مشترک‌اند. چه چیزی هنوز بینشان نشت می‌کند؟ (پاسخ: روی MQTT هیچ؛ registry نوع‌ها و singletonهای کنترلر هم per-process هستند. داوطلبی که فکر می‌کند و همین را می‌گوید، مدل را دارد.)
۶. دوست داشتی هوش مصنوعی چه چیزی را می‌دید یا می‌توانست انجام دهد که نمی‌توانست؟

راه‌حل‌های مرجع

نسخه‌های فشرده برای مصاحبه‌گر. هر پنج اصلاح با هم روی یک کپی از مخزن اعمال شد: شش تست موجود و mypy پاس می‌شوند و هر یک از باگ‌های جدول به رفتار مورد انتظار برمی‌گردد.

تسک ۰ · callback در paho

```
# I1820/mqtt/route.py
def on_connect(self, client, userdata, flags, reason_code, properties):
    ...
```

تسک ۰ · timestamp منجمد

```
# I1820/domain/log.py
def __init__(self, type, device, states, agent, timestamp=None):
    ...
    self.timestamp = timestamp or datetime.datetime.now(datetime.UTC)
```

تسک ۲ · فهرست فیلد برای هر کلاس

```
# I1820/things/base.py, inside AbstractThing.__new__ after the abstract check
for attr in ("statistics", "settings", "events", "states"):
    inherited = [f for base in bases for f in getattr(base, attr, [])]
    setattr(instance, attr, list(inherited))
for _, value in namespace.items():
    if isinstance(value, Field):
        getattr(instance, value.field_name).append(value)
```

تسک ۳ · last_seen

```
# I1820/discovery/discovery.py, in the else branch of ping()
stored = self._agents[agent.ident]
stored.last_seen = datetime.datetime.now(datetime.UTC)
if stored.things == agent.things:
    return
```

تسک ۳ · setting بدون مقدار

```
# I1820/things/fields/setting.py
def __get__(self, obj: Thing, _) -> T | None:
    return self.storage.get((obj.agent_id, obj.device_id))
```

برگرداندن None با Statistic و شکل پاسخ handler خواندن هم‌خوان است. ۴۰۴ هم قابل دفاع است اگر تیم ترجیح بدهد؛ نکته این است که انتخاب شده باشد، نه این‌که از یک KeyError به ارث رسیده باشد.

ملاحظات

- پنج باگ در محیط پایتون ۳.۱۲ با نسخه‌های قفل‌شده بازتولید شدند (pymongo 4.10.1، sanic 24.6.0، paho-mqtt 2.1.0). خرابی MQTT روی یک بروکر درون‌فرایندی بازتولید شد، نه EMQX.
- این‌که Mongo بین timestampهای مساوی دقیقاً چه مقداری برمی‌گرداند بررسی نشد. تله همچنان برقرار است چون معیار «مقدار بزرگ‌تر در فراخوانی دوم» به‌طور قابل اعتماد برآورده نمی‌شود؛ ولی رفتار واقعی را روی ماشین آماده‌سازی ببینید و اگر لازم شد صورت مسئله را تنظیم کنید.
- بودجه‌ی زمانی با فرض محیط آماده است. اگر داوطلب باید ایمیج یا وابستگی نصب کند، ۲۰ دقیقه به تسک ۰ اضافه کنید و آن زمان را نویز بدانید.